

[TOC]

Dashboard Source

This folder is used to keep the source for some of our Grafana dashboards, checked into, and managed by, git.

On **master** builds, the dashboards will be uploaded to <https://dashboards.gitlab.net>. Any local changes to these dashboards on the Grafana instance will be overwritten.

The dashboards are kept in **grafonnet** format, which is based on the jsonnet template language.

File nomenclature

We utilize the following file format: `dashboards/<service name, aka type>/<dashboard name>.dashboard.jsonnet`

Using this consistent schema makes URLs consistent, etc.

Example, the Container Registry is of service type **registry**. Therefore, `dashboards/registry/<somedashboard>.dashboard.jsonnet`

Extending Grafana dashboards

Video guide on how to extend dashboards

In order to extend Grafana dashboard you don't need to run Grafana locally. The most common scheme for extending dashboards is updating their definitions in your local repository and pushing changes to a testing playground on dashboards.gitlab.net.

An alternative way to check simple changes, that does not require installing dependencies on your local machine, is using a Grafana Playground folder. All users with viewer access to dashboards.gitlab.net, (ie, all GitLab team members), have full permission to edit all dashboards in the Playground Grafana folder. You can create dashboards in this folder using the Grafana Web UI.

If you, however, need to extend or modify an existing dashboard and create a merge request to persist these modification, you need be able to quickly create a snapshot of a new version of a dashboard to validate your changes. In order to do that you first need to install dependencies required by the `test-dashboard.sh` script. You will also need to obtain an API token for Grafana from 1Password.

Install dependencies

Follow the guidelines for setting up your development environment with `asdf` and required plugins as per the guidelines in the root README.MD for this repository.

- Ensure that you install `asdf` and plugins for `go-jsonnet` and `jsonnet-bundler`.
- Update vendor dependencies with `jb install`.
- Some people have found they couldn't use `./test-dashboard.sh` to create dashboards until they had installed and setup the 1Password CLI tool

Obtain the Grafana Playground API Key

We provide a Grafana API key through 1Password:

- Vault: `Engineering`
- Item: `Grafana playground API token`
- Field: `developer-playground-key` API Key

Load this key into the `GRAFANA_API_TOKEN` environment variable.

This expects the 1Password CLI tool to be installed:

```
op signin
export GRAFANA_API_TOKEN=$(op read "op://Engineering/Grafana playground API token/Tokens/dev")
```

This will be automatically done when using the `test-dashboard.sh` script. Alternatively, grab the API key from 1Password manually and set it with `export GRAFANA_API_TOKEN=...`

Modify a dashboard

In order to *modify* a dashboard you will need to write code using Grafonnet library built on top of Jsonnet syntax. In most cases you will also need to specify a PromQL query to source the data from Prometheus. You can experiment with PromQL using the Grafana playground for Prometheus.

Snapshots can only be created for dashboards, that have already been installed into Grafana. For entirely new dashboards, consider merging a basic dashboard (e.g. an empty one, see this example). Once this has been deployed, local edits can be tested as described above.

Create a new snapshot of the modified dashboard

1. To upload your dashboard, run `./test-dashboard.sh dashboard-folder-path/file.dashboard.json`. It will upload the file and return a link to your dashboard.
2. `./test-dashboard.sh -D $dashboard_path` will echo the dashboard JSON for pasting into Grafana.

Note that the playground and the snapshots are transient. By default, the snapshots will be deleted after 24 hours and the links will expire. Do not include links to playground dashboards in the handbook or other permanent content.

If `./test-dashboard.sh` is failing with a 403 error, try installing and setting up the 1Password CLI tool

Generating dashboards

To create dashboard JSONs, run `./dashboards/generate-dashboards.sh`. The generated manifests will be in the `/dashboards/generated` folder.

Dashboard Generation Process

The dashboard generation process converts jsonnet configuration files into JSON format that Grafana can consume:

1. **Create** dashboards using the jsonnet configuration language (`.dashboard.jsonnet` files)
2. **Process** jsonnet files to produce JSON output using the `jsonnet` library
3. **Upload** the resulting JSON files to Grafana (via API or UI)

Grafana only accepts JSON format, so the jsonnet-to-JSON conversion is a required step. The scripts in this repository automate this process for GitLab's infrastructure only.

Template Variables

Dashboards use Grafana template variables like “Environments”, “Stage”, and “Type” to create reusable dashboard templates across different contexts:

- **Environments:** e.g., `production`, `staging`, `development`
- **Stage:** e.g., `main`, `canary`
- **Type:** varies by use case

Organizations implementing these dashboards will need to adjust these variables to match their own environment structure.

Using Generated Dashboards

The generated dashboards provide metric visualizations for different aspects of the system (e.g., Runner performance, capacity, queuing). While the dashboards display the data, organizations should define their own:

- Alerting rules and thresholds
- Actions based on metrics
- Priorities based on their specific operational needs

Context for each dashboard can often be found in the description comments within the source jsonnet files.

Editing Files

- Dashboards should be kept in files with the following name: `/dashboards/[grafana_folder_name]/[name]`
 - `grafana_folder_name` refers to the grafana folder where the files will be uploaded to. Note that the folder must already be created.
 - These can be created via `./create-grafana-folder.sh <grafana_folder_name> <friendly_name>`
 - Example: `./create-grafana-folder.sh registry 'Container Registry'`
 - Note that if a folder already contains the name, it'll need to be removed or renamed in order for the API to accept the creation of a new folder
- Obtain a API key to the Grafana instance and export it in `GRAFANA_API_TOKEN`:
 - `export GRAFANA_API_TOKEN=123`
- To upload the files, run `./dashboards/upload.sh`

Shared Dashboard Definition Files

It's possible to generate multiple dashboards from a single, shared, jsonnet file.

The file should end with `.shared.jsonnet` and the format of the file should be as follows:

```
{
  "dashboard_uid_1": { /* Dashboard */ },
  "dashboard_uid_2": { /* Dashboard */ },
}
```

Protecting Dashboards From Deletion

By default we delete any dashboards that are not maintained within the `runbooks` repo.

If you have a dashboard you wish to maintain yourself via the UI or another means, you can exclude this from the deletion process.

Exclusions of protected dashboards are configured via the `protected-grafana-dashboards` file.

The easiest method is to add the label `protected` to your dashboard which will automatically exclude it.

You can alternatively add static dashboard uids or folder names as denoted in that file.

Backups

Dashboards that are not version-controlled in this repo are periodically purged from grafana. In order to recover a deleted dashboard (and commit it into this repo), you can look at the archive in grafana-dashboards.

The jsonnet docker image

- Google does not maintain official docker images for jsonnet.
- For this reason, we have a manual build step to build the `registry.gitlab.com/gitlab-com/runbooks/jsonnet` image.
- To update the image, run this job in the CI build manually